

KNN 实现鸢尾花分类实验指导

实验概览

实验目的

- 理解 **KNN** (K-Nearest Neighbors) 算法的基本原理和工作机制。
- 掌握使用 **KNN 算法** 对 **鸢尾花数据集** 进行分类的方法。
- 学会使用 **准确率** 等指标评估分类模型的性能。
- 掌握 **数据集划分** 方法及 **k 折交叉验证** 技术。
- 理解验证集在模型调参中的作用，掌握通过交叉验证选择最优 K 值的方法。

实验要求

- 掌握 Python 编程基础，熟悉 NumPy、Pandas 等科学计算库。
- 了解 scikit-learn 库的基本使用方法。
- 具备 **机器学习** 的基础理论知识，特别是 **分类算法** 的基本概念。

提交内容

实验报告（包含实验目的、步骤、结果分析等），采用 PDF 格式。报告应结构清晰、内容详实、语言流畅。

KNN 算法

KNN 算法是一种基于实例的学习方法，属于监督学习中的分类算法。其核心思想是计算待分类样本与训练集中样本的距离，找到距离最近的 K 个样本，并根据这 K 个样本的类别进行投票，将待分类样本归为票数最多的类别。

核心概念

- 距离度量**：常用的距离度量包括欧氏距离、曼哈顿距离等。
- K 值选择**：K 值的大小直接影响分类结果，通常通过交叉验证进行选择。
- 多数投票**：K 个最近邻中出现次数最多的类别即为预测类别。

算法步骤

- 计算待分类样本与所有训练样本的距离。
- 根据距离对训练样本进行排序。
- 选择距离最近的 K 个样本。
- 统计这 K 个样本中各类别的数量，将待分类样本归类为数量最多的类别。

算法优缺点

优点：算法简单、易于实现；无需训练过程，适用于多分类问题。

缺点：计算量大，特别是当训练集较大时；对高维数据效果不佳；

K 值的选择对结果影响较大。

深度学习的应用

- 分类问题**：鸢尾花分类、图像识别
- 推荐系统**：电影推荐、新闻推荐
- 相似性查询**：地理位置查询、病例检索
- 模式识别**：简单语音识别、行为识别

k 折交叉验证

k 折交叉验证是一种用于评估模型泛化性能的技术，尤其适用于数据量有限或需要更稳健性能估计的场景。在 k 折交叉验证中，首先将数据集划分为独立的训练集和测试集，然后在训练集上进行 k 折交叉验证以调优模型，最后在测试集上进行一次性能评估。

KNN 实现鸢尾花数据分类

1.数据准备

(1) 加载数据集

使用 Pandas 或 scikit-learn 的 datasets 模块加载鸢尾花数据集。

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris_df = load_iris()
```

(2) 数据预处理

将数据集划分为训练集（80%）和测试集（20%）。

```
# 拆分数据集
# 参数解释：特征, 标签, 拆分比例8:2, 随机种子
x_train, x_test, y_train, y_test = train_test_split(iris_df.data,
                                                    iris_df.target, test_size=0.2, random_state=11)
```

2.模型训练

(1) 导入 KNN 分类器

手动实现 KNN 分类器的函数。注意，请不要调用现成的 KNeighborsClassifier() 函数来完成本次实验。

```

# 分类器实现
class kNN(object):
    # 定义初始化方法,初始化kNN需要的超参数
    def __init__(self,n_neighbors = 1,dist_func = l1_distance):
        self.n_neighbors = n_neighbors
        self.dist_func = dist_func

    # 训练模型方法
    def fit(self,x,y):

    # 模型预测方法
    def predict(self,x):

```

(2) 创建 KNN 分类器实例

选择合适的 K 值（如 3、5、7 等），创建 KNN 类的实例。

```

# 定义一个knn实例
knn = kNN(n_neighbors = 3)

```

(3) 训练模型

使用训练集数据对 KNN 分类器进行训练，即调用 fit 方法，并使用 k 折交叉验证训练模型。

1. 折： 将训练集随机并均匀地分成 k 个互斥的子集。

2. 轮次： 进行 k 轮独立的训练-验证循环。

在每一轮中，取出 1 个折作为验证集（Validation Set）。

剩余的 k-1 个折合并作为训练集（Training Set）。

用训练集训练模型，并在验证集上评估，得到一个性能分数（如准确率）。

3. 聚合： 计算这 k 轮评估结果的平均值，作为模型的最终性能指标。

```

kf = KFold(n_splits=5, shuffle=True, random_state=35)
k_range = range(1, 11)
cv_scores = []

for k in k_range:
    fold_scores = []
    for train_idx, val_idx in kf.split(x_train):

```

3. 测试集评估网络

使用最优 K 值训练最终模型，并在测试集上进行性能评估，记录准确率等指标。

```
# 用最优k在测试集上训练并测试
knn = kNN(n_neighbors=best_k)
knn.fit(x_train, y_train)
y_pred = knn.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print('测试集上的预测准确率: ', accuracy)
```